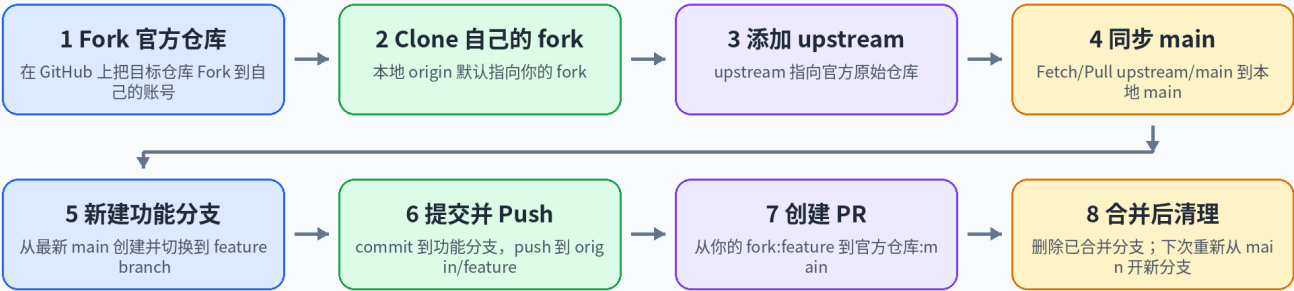


GitHub Fork 贡献流程指南

TortoiseGit 版：Fork、Clone、upstream、功能分支、PR 与合并后清理

GitHub Fork 贡献流程总览

核心原则：main 只负责同步官方最新代码；每个 PR 都从最新 main 新建独立功能分支。



一句话记忆：Fork 后本地有两个远端 - origin 是你的 fork，upstream 是官方仓库；开发永远从最新 main 开新分支。

适用场景：你将目标仓库 Fork 到自己的 GitHub 账号后，用 TortoiseGit 在本地开发，并向官方仓库提交 Pull Request。

核心结论：不要直接在 main 上开发；每个 PR 从最新 main 创建一个新的功能分支；PR 合并后可以删除该功能分支。

1. 先理解三个位置：origin、upstream、local

Fork 工作流里最容易混淆的是“代码到底在哪个仓库”。下面这张图把关系拆开：

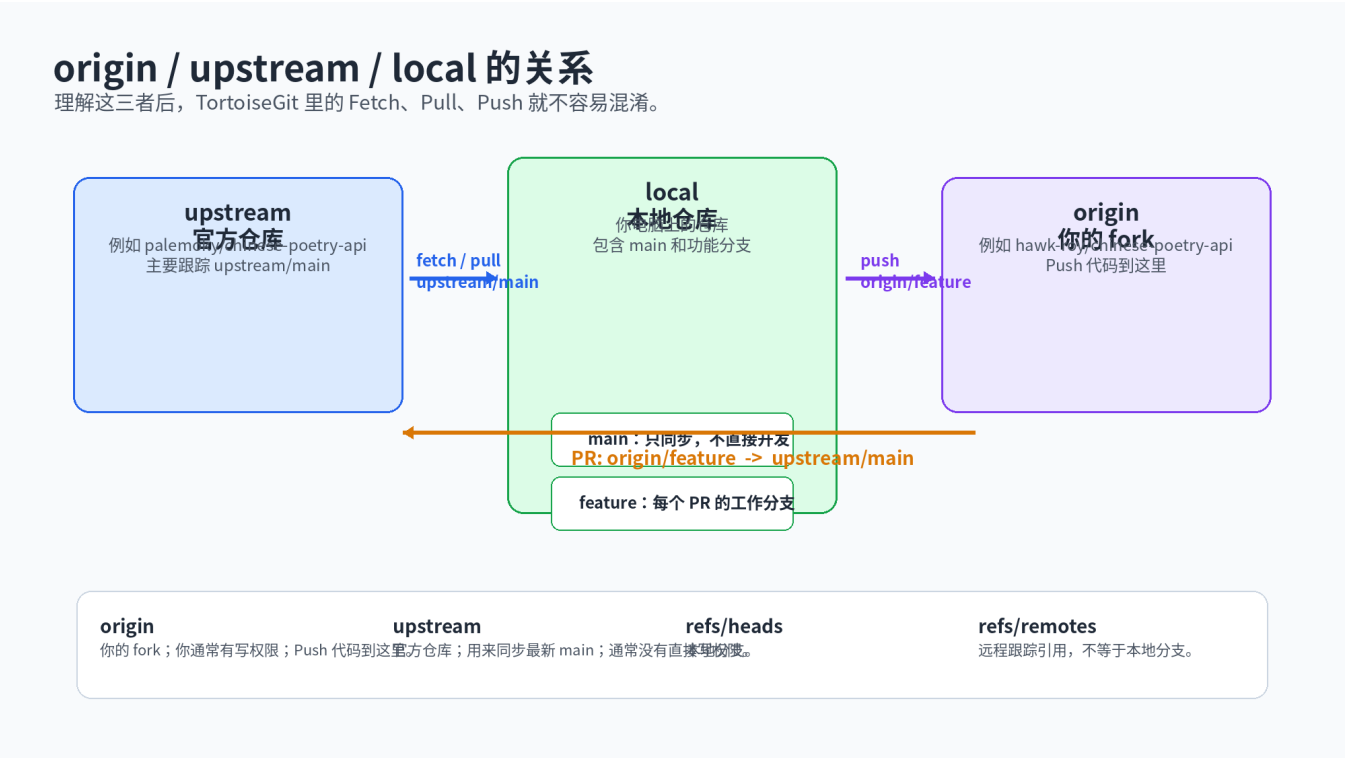


图 1: origin、upstream 与 local 的关系

名称	含义	你主要做什么
origin	你自己的 fork 仓库。	把本地功能分支 Push 到 origin，再从 origin/feature 发起 PR。
upstream	官方目标仓库。	每次开发前从 upstream/main 同步最新代码。
local	你电脑里的本地仓库。	本地 main 用于同步；功能分支用于实际开发。

关键点：upstream 是“整个本地仓库”的远端配置，不是某一个分支的属性。只是实际同步时，你通常拉的是 upstream/main。

2. 一次性配置流程

1. 在 GitHub 上 Fork 官方仓库。
2. 把你自己的 fork 克隆到本地。克隆后，origin 默认指向你的 fork。
3. 在 TortoiseGit 里添加官方仓库为 upstream。
4. 验证远端：origin = 你的 fork；upstream = 官方仓库。

命令行等价写法如下，便于你理解 TortoiseGit 背后的动作：

```
git clone <你的 fork 地址>
cd <repo>
git remote add upstream <官方仓库地址>
git remote -v
```

TortoiseGit 对应路径：右键项目文件夹 -> TortoiseGit -> Settings -> Git -> Remote。保留 origin，新建 upstream。

TortoiseGit 操作速查

下面是常用菜单路径。不同版本的中文翻译可能略有差异。



图 2: TortoiseGit 常用操作路径

3. 每次开始新任务的标准动作

每次开新 PR 前，都先把本地 main 同步到官方最新状态，然后从这个干净的 main 创建新分支。

1. Switch/Checkout 到 main。
2. Fetch upstream，拿到官方仓库的最新引用。
3. Pull 或 Merge upstream/main 到本地 main。
4. 可选：把更新后的本地 main Push 到 origin/main，让你的 fork 也跟上官方 main。
5. 从当前 main 创建一个新的功能分支，并切换到这个分支。

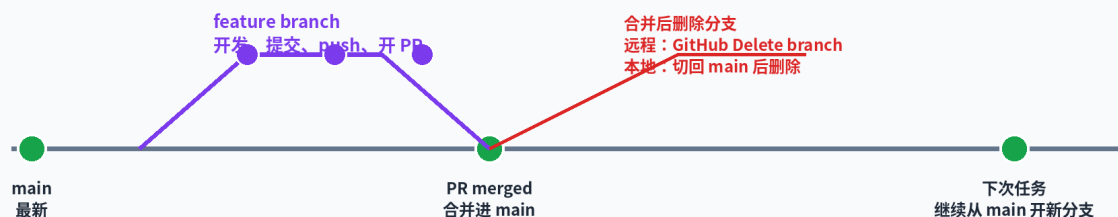
命令行等价写法：

```
git checkout main
git fetch upstream
git pull upstream main
git push origin main
git checkout -b my-new-feature
```

TortoiseGit 对应：Switch/Checkout 到 main -> Fetch 选择 upstream -> Pull 选择 upstream/main -> Create Branch 勾选切换到新分支。

一个功能分支的生命周期

已合并的功能分支可以删除；下次不要复用旧分支名，重新从最新 main 创建。



建议

每个 PR 用一个新分支；PR 合并后删除该分支；下一次从最新 main 再开新分支。

避免

不要在 main 上直接开发；不要为了新需求继续复用已经合并过的旧分支。

图 3：功能分支生命周期

4. 开发、提交、Push、创建 PR

在功能分支上修改代码，提交后 Push 到你的 fork，然后在 GitHub 上创建 PR。

1. 确认当前分支是功能分支，不是 main。
2. 修改代码。
3. Commit 到当前功能分支。
4. Push 到 origin/功能分支。
5. 在 GitHub 创建 PR：你的 fork:功能分支 -> 官方仓库:main。

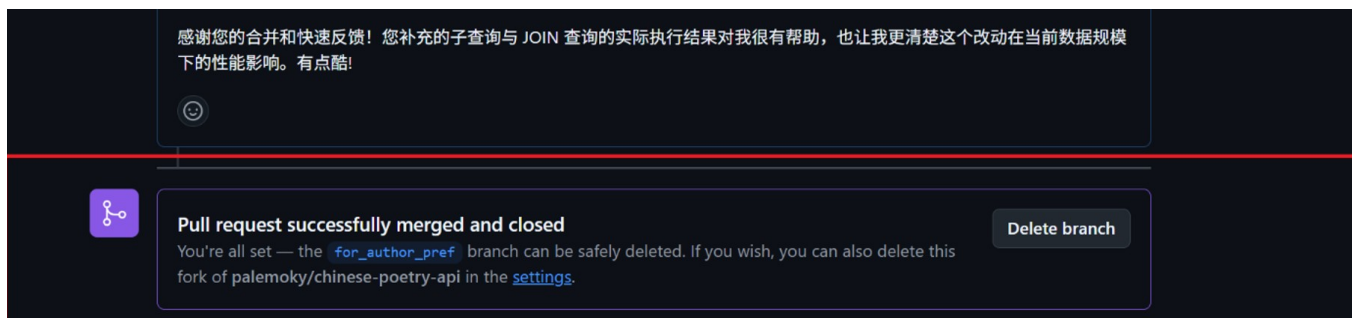
命令行等价写法：

```
git status
git add .
git commit -m "描述这次改动"
git push -u origin my-new-feature
```

PR 的方向要特别注意：base 是官方仓库 main；compare 是你 fork 里的功能分支。

5. PR 合并后的清理

PR 已经合并后，功能分支的任务就结束了。此时 GitHub 提示 Delete branch 是正常的。



图示：PR 合并后出现 Delete branch。它删除的是本次 PR 的源分支，不会撤销已合并到 main 的代码。

注意：旁边的 Revert 才是创建“撤销本次合并”的操作；清理分支时不要误点 Revert。

图 4：PR 合并后，GitHub 的 Delete branch 只删除源分支

- Delete branch 删除的是这次 PR 的源分支，例如 origin/for_author_pref。
- 它不会删除官方 main，也不会撤销已经合并进去的代码。
- 本地分支不会自动删除；需要你本地单独清理。
- 不要误点 Revert。Revert 是创建一个“撤销本次合并”的新 PR/提交。

6. 本地删除功能分支，以及 refs/remotes 的含义

如果你想删除本地功能分支，必须先切换到别的分支，通常是 main。

```
git checkout main
git branch -d for_author_pref
# 如果确认已合并但 Git 仍提示未合并，可用：
git branch -D for_author_pref
```

TortoiseGit 操作：Switch/Checkout 到 main -> Browse References -> refs/heads -> 右键本地功能分支 -> Delete Branch。

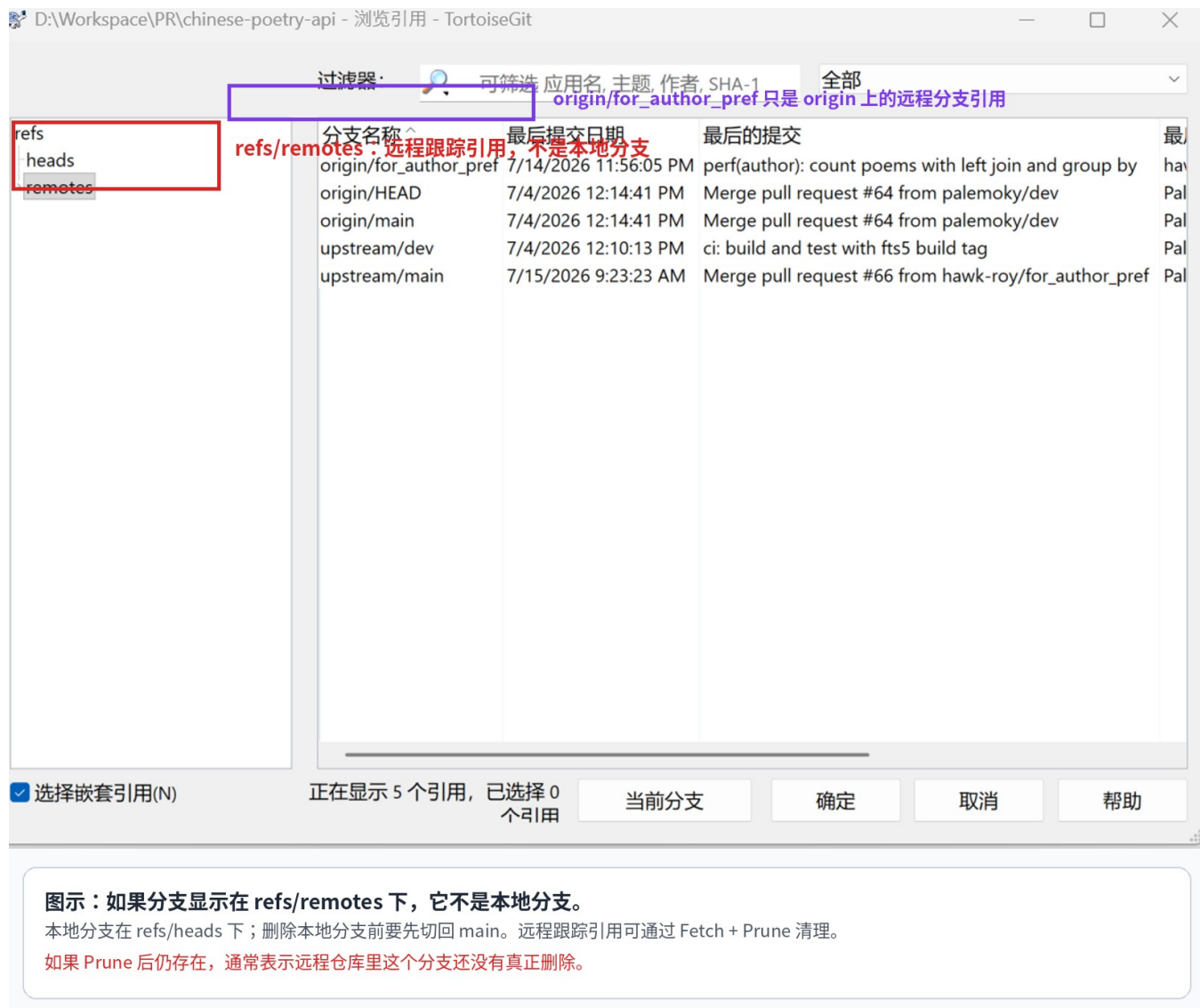


图 5: refs/remotes 下面的是远程跟踪引用，不是本地分支

如果看到 origin/for_author_pref，它通常只是远程跟踪引用。执行 Fetch 时勾选 Prune 可以清理已经在远程删除的引用。若 Prune 后仍存在，通常表示远程分支还没有真正删除。

7. 推荐的长期习惯

- main 只用于同步官方最新代码，不在 main 上直接开发。
- 每个任务、每个 PR，都从最新 main 新建一个功能分支。
- 功能分支名称描述本次改动，例如 fix-author-count、add-poem-tests。
- PR 合并后删除对应功能分支；下一次任务不要复用旧分支。

- 在 TortoiseGit 的 Push 窗口里确认本地分支和远端分支，不要把 main 当成功能分支误推。

8. 快速检查表

阶段	检查问题
准备开发前	我现在在 main 吗？main 已经从 upstream/main 同步了吗？
创建分支时	我是从最新 main 创建的新分支吗？分支名是否对应本次任务？
提交前	git status / TortoiseGit 状态里是否只包含本次任务相关文件？
Push 前	目标是不是 origin/我的功能分支，而不是 upstream/main？
开 PR 时	base 是否是官方仓库 main？compare 是否是我的 fork 功能分支？
合并后	是否可以删除远程功能分支？本地是否也要切回 main 后删除旧分支？

最后记住一句话：先同步 main，再开新分支；功能分支只服务一个 PR，合并后就可以清理。